Software Requirements

for

Smart Traffic Management System for Urban Congestion

Prepared by

Hannan Ahmed Qureshi (23100BTCSE14857)
Danish Khan (23100BTCSE14830)
Deepesh Kataria (23100BTCSE14834)

Under the guidance of

Prof. Virendra Dani

Department of Computer Science & Engineering
Shri Vaishnav Institute of Information Technology
Shri Vaishnav Vidyapeeth Vishwavidyalaya, Indore

Date created: 02 October 2026

Table of Contents

1. Introduction....4
1.1 Purpose....4
1.2 Document Conventions....4
1.3 Intended Audience and Reading Suggestions....4
1.4 Product Scope....4
1.5 References....5
2. Overall Description....5
2.1 Product Perspective....5
2.2 Product Functions....6
2.3 User Classes and Characteristics....7
2.4 Operating Environment....7
2.5 Design and Implementation Constraints....7
2.6 User Documentation....7
2.7 Assumptions and Dependencies....8
3. External Interface Requirements....8
3.1 User Interfaces....8
3.2 Hardware Interfaces....8
3.3 Software Interfaces....9
3.4 Communications Interfaces....9
4. System Features....10
4.1 Road Network and Traffic Scenario Simulation....10
4.2 Simulated Sensor Data Acquisition....11
4.3 Camera-Based Vehicle Detection (Computer Vision)....11
4.4 Traffic State Generation and Analysis....11
4.5 Traffic Prediction....12
4.6 Signal Controllers (Fixed-Time, Rule-Based, RL)....12
4.7 Emergency Vehicle Priority (Optional)....13
4.8 Performance Evaluation....14
4.9 Dashboard – Live Monitoring and Analytics....14
4.10 Dashboard – Signal Control and Manual Override....15
4.11 User Authentication and Authorisation....16
4.12 Data Storage and History....16
5. Other Nonfunctional Requirements....17
5.1 Performance Requirements....17
5.2 Safety Requirements....17
5.3 Security Requirements....18
5.4 Software Quality Attributes....18
5.5 Business Rules....18
6. Other Requirements....19
Appendix A: Glossary....19
Appendix B: Analysis Models....20

Revision History

Name	Date	Reason For Changes	Version
Hannan Ahmed Qureshi, Danish Khan, Deepesh Kataria	02-Oct-2026	Final SRS release	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Smart Traffic Management System for Urban Congestion (hereafter "STMS"), release 1.0. STMS is an AI-based software prototype that optimises traffic signal timings in an urban road network in order to reduce congestion and average commute time.

This SRS covers the complete prototype: the traffic simulation environment, the traffic data acquisition and analysis pipeline (including computer-vision-based vehicle detection), the AI signal controller, the performance-evaluation framework, the backend services, and the web dashboard used by traffic authorities to monitor and control signals. Physical deployment on live traffic hardware is outside the scope of release 1.0.

1.2 Document Conventions

- Requirements use the keyword "shall" for mandatory behaviour, "should" for desirable behaviour and "may" for optional behaviour.
- Each functional requirement has a unique tag of the form FR-<FEATURE>-<NN> (for example, FR-SIM-01). Non-functional requirements use NFR-<CATEGORY>-<NN>.
- Every requirement carries its own priority: High (essential for the prototype), Medium (important, to be delivered if schedule permits) or Low (optional enhancement).
- Terms in bold within descriptions are defined in Appendix A (Glossary).

1.3 Intended Audience and Reading Suggestions

Reader	Interest	Suggested sections
Project guide / evaluators	Scope, objectives, verification of the 10% target	1, 2, 4.8, 5.1, Appendix B
Developers (simulation, AI, backend, frontend)	Detailed functional and interface requirements	2, 3, 4, 5
Testers	Verifiable requirements and acceptance criteria	4, 5, Appendix B
Traffic authority representatives (prospective users)	What the dashboard and controls offer	2.2, 2.3, 3.1, 4.9, 4.10

The document is organised as follows: Section 1 introduces the project; Section 2 gives an overall description of the product; Section 3 defines external interfaces; Section 4 specifies system features and their functional requirements; Section 5 specifies non-functional requirements; Section 6 lists other requirements. The appendices contain a glossary and analysis models (UML diagrams). Readers new to the project should read Sections 1 and 2 first.

1.4 Product Scope

Urban traffic signals commonly run on fixed timing plans that cannot react to changing demand, causing queues, long waiting times and wasted fuel. STMS addresses this by observing traffic in real time,

converting observations into a standard traffic state, and using a reinforcement-learning controller to decide which signal phase should be green and for how long.

Objectives:

- Reduce average commute (travel) time by at least 10% compared with a fixed-time signal baseline in a simulated urban environment built from real OpenStreetMap data.
- Analyse real-time traffic data from simulated sensors and from camera video (OpenCV + YOLO) to estimate vehicle counts, queues and congestion.
- Provide a dashboard through which traffic authorities can monitor traffic and signal status and manually control signals.
- Compare three control strategies on identical traffic: fixed-time, rule-based adaptive, and reinforcement-learning-based.

Out of scope for release 1.0: deployment to physical signal controllers, procurement or installation of cameras and IoT hardware, and city-wide coverage beyond the selected simulation area.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications; ISO/IEC/IEEE 29148:2018, Requirements engineering.
- SRS Template – SVIIT (institute-provided template followed by this document).
- Eclipse SUMO – Simulation of Urban MObility documentation, https://sumo.dlr.de/docs/ (including TraCI and OpenStreetMap import / netconvert).
- OpenStreetMap data and licence (ODbL), https://www.openstreetmap.org.
- Ultralytics YOLO documentation, https://docs.ultralytics.com; OpenCV documentation, https://docs.opencv.org.
- Gymnasium documentation, https://gymnasium.farama.org; Stable-Baselines3 documentation, https://stable-baselines3.readthedocs.io.
- J. Schulman et al., “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- FastAPI documentation, https://fastapi.tiangolo.com; Flutter web documentation, https://docs.flutter.dev/platform-integration/web.
- Project concept note and technical plan , Internal Project Document, team internal, 2026.

2. Overall Description

2.1 Product Perspective

STMS is a new, self-contained prototype; it does not replace an existing software product. It is designed so that its AI controller depends only on a standard Traffic State, regardless of whether that state is produced by simulator sensors or by computer-vision analysis of camera video. This separation allows the controller to be trained quickly against the simulator and later demonstrated with camera footage.

The system consists of the following major components and external interfaces:

Component	Role	Technology
Traffic Simulation	Virtual urban road network (from OpenStreetMap), vehicles, intersections, signals, scenarios	SUMO, netconvert, TraCI

Component	Role	Technology
Data Acquisition	Simulated sensors and camera-based vehicle detection	SUMO detectors, OpenCV, YOLO
Traffic Analysis	Vehicle counts, queues, density, waiting time → Traffic State	Python
AI / ML Engine	Rule-based and RL controllers; optional traffic prediction model	Gymnasium, PPO (Stable-Baselines3), LSTM/GRU/XGBoost (optional)
Signal Control	Applies decisions to simulated signals through TraCI	Python, TraCI
Evaluation Module	Runs fixed-time vs. rule-based vs. RL on identical scenarios and computes metrics	Python
Backend	REST and WebSocket bridge between AI/simulator and dashboard; persistence	FastAPI, WebSocket, PostgreSQL/Supabase
Dashboard	Monitoring, analytics and manual control for traffic authorities	Flutter Web

External interfaces are the SUMO simulator, the video/camera source (recorded traffic footage in release 1.0), the database server, and the web browser of the dashboard user. Architecture, component and deployment views are given in Appendix B.

2.2 Product Functions

- Import a real-world road network from OpenStreetMap and generate vehicle demand for multiple traffic scenarios (low, medium, heavy, rush hour, directional congestion).
- Collect traffic data from simulated sensors and from camera video using OpenCV and YOLO vehicle detection.
- Convert raw observations into a standard Traffic State (counts, queues, waiting times, current phase, phase duration).
- Run a fixed-time baseline controller, a rule-based adaptive controller, and a reinforcement-learning (PPO) controller.
- Optionally predict short-term traffic growth per approach (for example the next 60 seconds).
- Apply signal decisions to the simulator in a continuous observe-decide-act loop.
- Evaluate controllers and report travel time, waiting time, queue length, speed, throughput and percentage improvement.
- Display live traffic, signal status, statistics and historical charts on a web dashboard.
- Allow authorised users to switch between AI and manual mode and override signals, including an emergency override.
- Store historical traffic and decision data and authenticate users by role.
- Optional: emergency-vehicle priority, multi-intersection coordination, congestion prediction display, route optimisation.

2.3 User Classes and Characteristics

User class	Characteristics	Functions used	Importance
Traffic Operator	Traffic authority staff; moderate technical skill; frequent use	Monitor dashboard, view statistics, switch to manual mode, change/extend signal, emergency override	High
Traffic Administrator	Senior authority user; manages accounts and policies	All operator functions, user management, configuration, historical reports	High
Researcher / Developer	Team members and evaluators; high technical skill	Create scenarios, train and evaluate controllers, run experiments, export results	High
Emergency Services (indirect)	Do not use the system directly	Benefit from emergency priority mode (optional feature)	Low
Commuters (indirect)	Beneficiaries of reduced travel time	None directly	Low

2.4 Operating Environment

- Simulation, AI and backend: Windows 10/11 or Ubuntu 22.04 or later; Python 3.10 or later; SUMO (current stable release) with TraCI.
- Training machine: multi-core CPU with at least 16 GB RAM; a CUDA-capable GPU is recommended for YOLO and optional for PPO.
- Database: PostgreSQL 14 or later, or a Supabase-hosted PostgreSQL instance.
- Dashboard: Flutter Web, running in current versions of Google Chrome, Microsoft Edge and Mozilla Firefox.
- Network: TCP/IP connectivity between dashboard, backend and AI controller (local network or localhost for the prototype).

2.5 Design and Implementation Constraints

- The road network shall be derived from OpenStreetMap data; map licence (ODbL) attribution is mandatory.
- SUMO is the mandated traffic simulator; the controller shall interact with it through TraCI.
- Python shall be the primary language for simulation control, AI and backend; Flutter (Dart) for the dashboard.
- The RL environment shall comply with the Gymnasium API; YOLO shall not be placed in the RL training loop, to keep training tractable.
- The project is an academic prototype with a fixed timeline and a small team; scope is limited to a single area and a small number of intersections.
- Only open-source or free-tier tools shall be used.
- Footage used for computer vision shall be publicly available or self-recorded, with licence terms respected.

2.6 User Documentation

- User manual for the dashboard (PDF), including screenshots and role-wise instructions.

- Setup and installation guide (SUMO, Python environment, backend, dashboard).
- Developer guide covering architecture, REST/WebSocket API reference and training/evaluation procedure.
- In-dashboard tooltips and help page.

2.7 Assumptions and Dependencies

Assumptions:

- A suitable area of a real city will be chosen and its OpenStreetMap data converts into a valid SUMO network with traffic lights.
- Real traffic counts for the chosen area may not be available; synthetic demand patterns will approximate real conditions.
- Recorded traffic video of adequate quality is available for the computer-vision demonstration.
- Results obtained in simulation are representative for the purpose of demonstrating the 10% improvement; they are not a guarantee of real-world performance.

Dependencies:

- SUMO, TraCI, OpenCV, Ultralytics YOLO, Gymnasium, Stable-Baselines3, PyTorch, FastAPI, Flutter SDK, PostgreSQL/Supabase.
- OpenStreetMap data availability.
- Pretrained YOLO weights (fine-tuning on a vehicle dataset is optional).

3. External Interface Requirements

3.1 User Interfaces

The dashboard is a responsive web application with the following screens. Standard elements on every screen: header with system name, logged-in user and role, navigation menu, system-status indicator (simulator connected / AI mode), and a logout control. Errors are shown as non-blocking messages with a plain-language explanation and a suggested action.

Screen	Content
Login	Username/password; role-based redirect
Live Dashboard	Intersection view with signal colours per approach, countdown, vehicle count/queue per approach, congestion level, average waiting time, throughput, current control mode
Analytics	Fixed-time vs. rule-based vs. AI comparison (travel time, waiting time, queue length, throughput), percentage improvement
Signal Control	Mode selector (AI Automatic / Manual), extend green, change phase, emergency override, confirmation dialogs
History	Time-series charts of vehicle counts, waiting time, speed, queue length and AI decisions with date/intersection filters
Administration	User and role management, system configuration, audit log

Detailed wireframes will be maintained in a separate user-interface specification.

3.2 Hardware Interfaces

- Release 1.0 has no direct interface to physical signal controllers; signals are simulated in SUMO.
- Camera input: video files (MP4/AVI) or an RTSP/HTTP video stream readable by OpenCV. Resolution of at least 720p and frame rate of at least 15 FPS recommended.
- Optional IoT sensors (inductive loops, radar) are represented by SUMO detectors (induction loops, lane-area detectors) that mimic real sensor output.
- Compute hardware (CPU/GPU) is accessed through the operating system and PyTorch/CUDA drivers.

3.3 Software Interfaces

Interface	Version / Type	Data in → out	Purpose
SUMO via TraCI	Current stable SUMO; TraCI Python API	Out: signal phase and duration commands; In: vehicle counts, queue, speed, waiting time, travel time	Simulation control and observation
OpenStreetMap / netconvert	OSM XML export	In: map file → Out: SUMO network	Road network generation
OpenCV + YOLO (Ultralytics)	Python packages	In: video frames → Out: detected vehicle classes, counts, density	Camera-based traffic data
Gymnasium / Stable-Baselines3	Python packages	Observation, action, reward; trained model file	RL training and inference
PostgreSQL / Supabase	SQL over TCP	Out: records of traffic data, decisions, metrics; In: history queries	Persistence
FastAPI backend	REST (JSON) + WebSocket	See Section 3.4	Bridge between AI/simulator and dashboard

3.4 Communications Interfaces

The dashboard communicates with the backend over HTTP/HTTPS (REST, JSON) and WebSocket for live updates. The backend communicates with the AI controller in-process or over localhost. Indicative endpoints:

Method / Path	Description
POST /auth/login	Authenticate user; returns token
GET /traffic/current	Current traffic state of all (or one) intersections
GET /intersection/{id}	Detail for one intersection
GET /signals	Current signal state and remaining seconds
POST /signal/override	Manual signal command (authorised roles only)
POST /mode	Switch between AI automatic and manual modes
GET /statistics	Aggregated metrics and controller comparison

Method / Path	Description
GET /history	Historical records with filters
WS /live	Server push of traffic state and signal changes (target period 1 s)

All control and authentication traffic shall be protected with token-based authentication and TLS when deployed beyond localhost. Messages are JSON; timestamps are ISO 8601 (UTC).

4. System Features

Functional requirements are organised by system feature. Features follow the pipeline: simulation → data acquisition → analysis → AI → control → evaluation → backend → dashboard.

4.1 Road Network and Traffic Scenario Simulation

4.1.1 Description and Priority

Builds the virtual urban environment from a real OpenStreetMap area in SUMO, with intersections, signals and configurable vehicle demand. This is the foundation for training, evaluation and the demonstration. Priority: High.

4.1.2 Stimulus/Response Sequences

- Researcher selects an OSM area / map file → system generates the SUMO network and reports success or errors.
- Researcher chooses a scenario → system generates vehicle routes and starts the simulation.

4.1.3 Functional Requirements

ID	Requirement	Priority
FR-SIM-01	The system shall convert an OpenStreetMap extract into a SUMO road network (roads, lanes, speed limits, junctions, traffic lights).	High
FR-SIM-02	The system shall allow selection of one or more signalised intersections of the network as controlled intersections.	High
FR-SIM-03	The system shall generate vehicle demand for the scenarios: Balanced, Directional congestion (N/S/E/W), Low, Medium, Heavy and Rush hour.	High
FR-SIM-04	The system shall support randomised demand using a configurable random seed so that runs are reproducible.	High
FR-SIM-05	The system shall support multiple vehicle types (car, bus, truck, motorcycle) with configurable proportions.	High
FR-SIM-06	The system shall report an error and refuse to start if the network or route files are invalid.	High
FR-SIM-07	The system shall run the simulation with and without the SUMO GUI (headless mode for training).	Medium

4.2 Simulated Sensor Data Acquisition

4.2.1 Description and Priority

Retrieves per-lane traffic measurements from the simulator through TraCI, acting as the virtual IoT sensor network. Priority: High.

4.2.2 Stimulus/Response Sequences

- Simulation advances one control step → system polls detectors and lanes and passes measurements to Traffic Analysis.

4.2.3 Functional Requirements

ID	Requirement	Priority
FR-SEN-01	The system shall connect to SUMO through TraCI and manage the connection lifecycle (start, step, close).	High
FR-SEN-02	The system shall retrieve, for each incoming lane/approach: vehicle count, halting (queued) vehicles, mean speed and accumulated waiting time.	High
FR-SEN-03	The system shall retrieve the current signal phase and elapsed phase duration of each controlled intersection.	High
FR-SEN-04	The system shall record per-vehicle travel time and waiting time on trip completion.	High
FR-SEN-05	The system shall handle loss of the TraCI connection by logging the error and stopping the run safely.	High

4.3 Camera-Based Vehicle Detection (Computer Vision)

4.3.1 Description and Priority

Processes traffic video using OpenCV and YOLO to detect and count vehicles per approach and estimate density. It produces the same Traffic State format as the simulator sensors. It is demonstrated on recorded footage and is not part of RL training. Priority: Medium.

4.3.2 Stimulus/Response Sequences

- Operator/Researcher selects a video source and defines approach regions → system displays detections and per-approach counts in near real time.

4.3.3 Functional Requirements

ID	Requirement	Priority
FR-CV-01	The system shall read frames from a video file or a stream using OpenCV.	High
FR-CV-02	The system shall detect vehicles and classify them as car, bus, truck or motorcycle using a YOLO model.	High
FR-CV-03	The system shall allow the user to define one region of interest per approach (N, S, E, W) on the video frame.	High
FR-CV-04	The system shall count vehicles per approach and compute a traffic density level (Low, Medium, High) using configurable thresholds.	High
FR-CV-05	he system shall map camera-derived measurements into the common Traffic State format and shall indicate fields that cannot be directly obtained from video.	High

ID	Requirement	Priority
FR-CV-06	The system shall display annotated frames (bounding boxes and counts) on request.	Low
FR-CV-07	The system shall report detection accuracy (precision/recall) on a labelled test clip as part of evaluation.	Low

4.4 Traffic State Generation and Analysis

4.4.1 Description and Priority

Transforms raw sensor or camera data into a source-independent Traffic State used by all controllers and the dashboard. Priority: High.

4.4.2 Stimulus/Response Sequences

- New measurements arrive → system computes the Traffic State and congestion level and publishes it.

4.4.3 Functional Requirements

ID	Requirement	Priority
FR-STA-01	The system shall construct a Traffic State containing, per approach: vehicle count, waiting time, queue length, mean speed; and per intersection: current phase and phase duration.	High
FR-STA-02	The system shall compute the congestion level (Low/Medium/High) per approach and intersection using configurable thresholds.	High
FR-STA-03	The system shall compute total waiting vehicles, average waiting time and throughput (vehicles/min).	High
FR-STA-04	The system shall convert data from simulator sensors and camera-based detection into a common Traffic State interface. Fields unavailable directly from a camera source shall be estimated where feasible or marked unavailable.	High
FR-STA-05	The system shall normalise the Traffic State into a numeric vector for use as RL observation.	High

4.5 Traffic Prediction

4.5.1 Description and Priority

Predicts short-term traffic growth on each approach using a machine-learning model, to support proactive decisions and dashboard alerts. This is separate from the RL controller. Priority: Medium.

4.5.2 Stimulus/Response Sequences

- System has a window of recent Traffic States → predicts vehicle count/congestion for the next horizon (for example 60 s).

4.5.3 Functional Requirements

ID	Requirement	Priority
FR-PRD-01	The system shall train a prediction model (LSTM, GRU, Random Forest or XGBoost) on simulator-generated data.	High
FR-PRD-02	The system shall output predicted vehicle count and congestion level per approach for a configurable horizon.	High
FR-PRD-03	The system shall report prediction error (MAE/RMSE) on a held-out dataset.	High
FR-PRD-04	The system shall display predicted congestion on the dashboard.	Low
FR-PRD-05	The system should be able to feed the prediction as an additional input to the RL controller.	Low

4.6 Signal Controllers (Fixed-Time, Rule-Based, RL)

4.6.1 Description and Priority

Provides three interchangeable controllers that decide signal phase and duration from the Traffic State: a fixed-time baseline, a rule-based adaptive controller and a PPO reinforcement-learning controller. The RL controller is the core AI component. Priority: High.

4.6.2 Stimulus/Response Sequences

- Controller receives a Traffic State → returns an action (keep phase, switch, extend green by 5 s, extend green by 10 s).
- Researcher starts training → system trains PPO agent over many simulated episodes and saves the model.

4.6.3 Functional Requirements

ID	Requirement	Priority
FR-CTL-01	The system shall implement a fixed-time controller with configurable green and yellow durations (default 30 s green, 3 s yellow).	High
FR-CTL-02	The system shall implement a rule-based adaptive controller that extends or switches green based on queue length and waiting time thresholds.	High
FR-CTL-03	The system shall provide a Gymnasium environment (TrafficEnvironment) wrapping SUMO with defined observation space, discrete action space and reward.	High
FR-CTL-04	The reward function shall penalise waiting time and queue length and reward vehicles passed; the exact formula shall be configurable.	High
FR-CTL-05	The system shall train a PPO agent with Stable-Baselines3, supporting randomised scenarios and seeds, and save and load trained models.	High
FR-CTL-06	The system shall enforce minimum green time, mandatory yellow and all-red clearance intervals on every phase change regardless of controller output.	High
FR-CTL-07	The system shall apply controller decisions to SUMO through TraCI and then observe the resulting state (closed control loop).	High
FR-CTL-08	The system shall allow the active controller to be selected per run (Fixed, Rule-based, RL).	High

ID	Requirement	Priority
FR-CTL-09	The system shall log training progress (episode reward, mean waiting time) for plotting.	High
FR-CTL-10	The system should support coordination of multiple intersections.	Low

4.7 Emergency Vehicle Priority (Optional)

4.7.1 Description and Priority

Gives priority to an approach on which an emergency vehicle is present. Implemented only after the core system works. Priority: Low.

4.7.2 Stimulus/Response Sequences

- Emergency vehicle detected (simulated vehicle type or YOLO class) → system grants green to its approach and notifies the dashboard.

4.7.3 Functional Requirements

ID	Requirement	Priority
FR-EMG-01	The system shall detect emergency vehicles in the simulation by vehicle type.	Low
FR-EMG-02	The system shall switch the relevant approach to green after safe clearance and restore normal control afterward.	Low
FR-EMG-03	The system shall record each emergency-priority event.	Low

4.8 Performance Evaluation

4.8.1 Description and Priority

Compares controllers on identical traffic to demonstrate the target of at least 10% reduction in average commute time. Priority: High.

4.8.2 Stimulus/Response Sequences

- Researcher launches an experiment → system runs each controller on the same scenarios and seeds and produces a comparison report.

4.8.3 Functional Requirements

ID	Requirement	Priority
FR-EVL-01	The system shall run each selected controller on identical scenarios, demand and random seeds.	High
FR-EVL-02	The system shall compute average travel time, average waiting time, average queue length, average speed, throughput and congestion level.	High
FR-EVL-03	The system shall compute percentage improvement = (baseline − AI) / baseline × 100 for each metric.	High
FR-EVL-04	The system shall repeat each experiment over multiple seeds (at least 5) and report mean and standard deviation.	High
FR-EVL-05	The system shall indicate whether the 10% travel-time reduction target has been met.	High

ID	Requirement	Priority
FR-EVL-06	The system shall export results as CSV and charts (PNG).	High
FR-EVL-07	The system shall compare Fixed-time, Rule-based adaptive and RL controllers in one report.	High

4.9 Dashboard – Live Monitoring and Analytics

4.9.1 Description and Priority

Web dashboard that presents live traffic conditions, signal status and statistics to traffic authorities. Priority: High.

4.9.2 Stimulus/Response Sequences

- User logs in → dashboard opens live view and receives updates every second.
- User opens Analytics/History → system shows charts for the chosen intersection and period.

4.9.3 Functional Requirements

ID	Requirement	Priority
FR-DSH-01	The dashboard shall display, per intersection, signal state (red/yellow/green) and remaining seconds for each approach.	High
FR-DSH-02	The dashboard shall display vehicle count, queue length and congestion level per approach.	High
FR-DSH-03	The dashboard shall display average waiting time, average travel time, throughput and current improvement percentage versus baseline.	High
FR-DSH-04	The dashboard shall update live data at least once per second via WebSocket.	High
FR-DSH-05	The dashboard shall show a comparison chart of Fixed vs. AI controllers.	High
FR-DSH-06	The dashboard shall show historical time-series for vehicles, waiting time, speed, queue length and AI decisions with intersection and time filters.	High
FR-DSH-07	The dashboard shall show the current control mode (AI or Manual) and the last AI decision.	High
FR-DSH-08	The dashboard shall indicate loss of connection and stale data clearly.	High
FR-DSH-09	The dashboard shall display alerts when congestion level is High.	Medium

4.10 Dashboard – Signal Control and Manual Override

4.10.1 Description and Priority

Allows authorised users to take manual control of signals, extend or change phases and trigger an emergency override. Priority: High.

4.10.2 Stimulus/Response Sequences

- Operator selects Manual mode → system suspends AI control of the selected intersection.
- Operator issues change/extend command → system validates safety rules, applies command through the backend and confirms.

4.10.3 Functional Requirements

ID	Requirement	Priority
FR-OVR-01	The system shall allow Operators and Administrators to switch each intersection between AI Automatic and Manual mode.	High
FR-OVR-02	In Manual mode, the user shall be able to extend the current green or switch to the next phase.	High
FR-OVR-03	The system shall validate every manual command against safety rules (FR-CTL-06) and reject unsafe commands with an explanatory message.	High
FR-OVR-04	The system shall provide an Emergency Override that sets a chosen approach to green after safe clearance.	High
FR-OVR-05	The system shall require confirmation before executing overrides.	High
FR-OVR-06	The system shall write every control action (user, time, intersection, action, result) to an audit log.	High
FR-OVR-07	The system shall return to AI control on user request or after a configurable manual timeout.	High

4.11 User Authentication and Authorisation

4.11.1 Description and Priority

Secures the dashboard and ensures only permitted roles can control signals. Priority: High.

4.11.2 Stimulus/Response Sequences

- User submits credentials → system authenticates and issues a session token with role.

4.11.3 Functional Requirements

ID	Requirement	Priority
FR-AUT-01	The system shall authenticate users with username and password.	High
FR-AUT-02	The system shall enforce role-based access: Operator (monitor, control), Administrator (all plus user management), Viewer (monitor only).	High
FR-AUT-03	The system shall store passwords only as salted hashes.	High
FR-AUT-04	The system shall expire inactive sessions after a configurable time (default 30 minutes).	High
FR-AUT-05	The system shall allow Administrators to create, disable and change roles of users.	Medium

4.12 Data Storage and History

4.12.1 Description and Priority

Persists traffic data, decisions and experiment results for analytics and auditing. Priority: Medium.

4.12.2 Stimulus/Response Sequences

- Each control step → system stores a record. User requests history → system queries and returns data.

4.12.3 Functional Requirements

ID	Requirement	Priority
FR-DAT-01	The system shall store timestamp, intersection id, vehicle counts, waiting time, queue length, signal phase, travel time, AI action and congestion level.	High
FR-DAT-02	The system shall store experiment configurations and results for later comparison.	High
FR-DAT-03	The system shall provide history queries filtered by intersection and time range.	High
FR-DAT-04	The system shall retain data for a configurable period (default 30 days) and allow export to CSV.	Low

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement	Priority
NFR-PRF-01	The RL controller shall return an action within 100 ms of receiving a Traffic State (excluding simulator step time).	High
NFR-PRF-02	Live dashboard data shall be at most 2 seconds old under normal load.	High
NFR-PRF-03	REST endpoints other than history queries shall respond within 500 ms for up to 10 concurrent users.	High
NFR-PRF-04	The vehicle detection pipeline shall process at least 10 frames per second on the reference GPU machine, or at least 3 FPS on CPU only.	Medium
NFR-PRF-05	Headless simulation shall run faster than real time (at least 5x) on the reference machine for the selected network.	Medium
NFR-PRF-06	Overall result: the RL controller shall reduce average travel time by at least 10% versus the fixed-time baseline, averaged over the evaluation scenarios and seeds.	High

5.2 Safety Requirements

- The system shall never display or command conflicting green phases for crossing movements.
- Every green-to-red transition shall include a yellow interval (default 3 s) and all-red clearance where configured; neither AI nor manual commands may bypass this.
- A minimum and maximum green time shall be enforced for every phase to prevent starvation of any approach.
- On failure of the AI controller or loss of connection, the system shall fall back automatically to the fixed-time plan and alert the operator.
- Simulation results shall be labelled as simulated; the system shall not be connected to real signal controllers without a separate safety certification process.

5.3 Security Requirements

- All dashboard access shall require authentication; control functions require Operator or Administrator role.
- Passwords shall be stored as salted hashes; tokens shall expire.
- Communication outside localhost shall use HTTPS/WSS.
- Input to all APIs shall be validated to prevent injection and malformed commands.
- All control actions shall be logged in a tamper-evident audit log. Camera footage containing identifiable persons or number plates shall not be stored beyond processing; only aggregated counts are retained.

5.4 Software Quality Attributes

Attribute	Requirement
Reliability	System shall run a 1-hour simulation session without crash; backend uptime of at least 99% during demonstration periods.
Usability	A trained operator shall be able to switch to manual mode and extend a green phase within 3 clicks and 30 seconds.
Maintainability	Modular design with controller, data source and environment behind clear interfaces; PEP 8 compliance; documented code.
Portability	Runs on Windows and Ubuntu; dashboard works in current Chrome, Edge and Firefox.
Interoperability	Controllers consume the standard Traffic State and are independent of data source.
Extensibility	New controllers, intersections or maps can be added without modifying the dashboard code.
Reproducibility	Experiments are reproducible using stored seeds and configuration.
Testability	Unit tests for analysis and controllers; integration tests for the API and TraCI loop.

Relative preference: safety and correctness take priority over performance; reliability and reproducibility take priority over visual polish.

5.5 Business Rules

- Only Operators and Administrators may switch to Manual mode or issue overrides; Viewers have read-only access.
- Only Administrators may manage users and system configuration.
- Emergency override takes precedence over AI and manual commands, subject to safety constraints.
- The improvement claim shall be computed only from runs of controllers on identical scenarios and seeds.
- OpenStreetMap-derived data shall be attributed per the ODbL licence.

6. Other Requirements

- Database: relational schema with tables for intersections, traffic_records, signal_events, decisions, experiments, users, audit_log (see ER diagram in Appendix B).
- Version control: source code in Git/GitHub with branching for team development.
- Legal: use of map data, video datasets and pretrained models shall comply with their licences.
- Internationalisation: dashboard text in English; units metric (km/h, metres, seconds).
- Reuse: the controller and Traffic State interface shall be reusable for other maps without code change.
- Delivery: source code, trained model, scenario files, documentation and a demonstration script.

Appendix A: Glossary

Term	Definition
STMS	Smart Traffic Management System
SUMO	Simulation of Urban MObility – open-source microscopic traffic simulator
TraCI	Traffic Control Interface – SUMO's API for online control and observation
OSM	OpenStreetMap – open map database
OpenCV	Open-source computer vision library
YOLO	You Only Look Once – real-time object detection model family
RL	Reinforcement Learning
PPO	Proximal Policy Optimization – policy-gradient RL algorithm
Gymnasium	Standard Python API for RL environments
Traffic State	Standard numeric/structured description of current traffic at an intersection
Phase	A combination of signal indications that give right of way to a set of movements
Fixed-time controller	Signal controller using a pre-set timing plan
Queue length	Number of halted vehicles waiting at an approach
Throughput	Vehicles passing an intersection per unit time
Approach	An incoming road direction to an intersection (N, S, E, W)
Episode	One complete simulation run used for RL training
Reward	Numerical feedback telling the RL agent how good an action was
REST / WebSocket	HTTP request-response API style / persistent two-way connection for live data
IoT	Internet of Things – networked sensors
ODbL	Open Database Licence (used by OpenStreetMap)
MAE / RMSE	Mean Absolute Error / Root Mean Square Error

Appendix B: Analysis Models

B.1 – Class Diagram

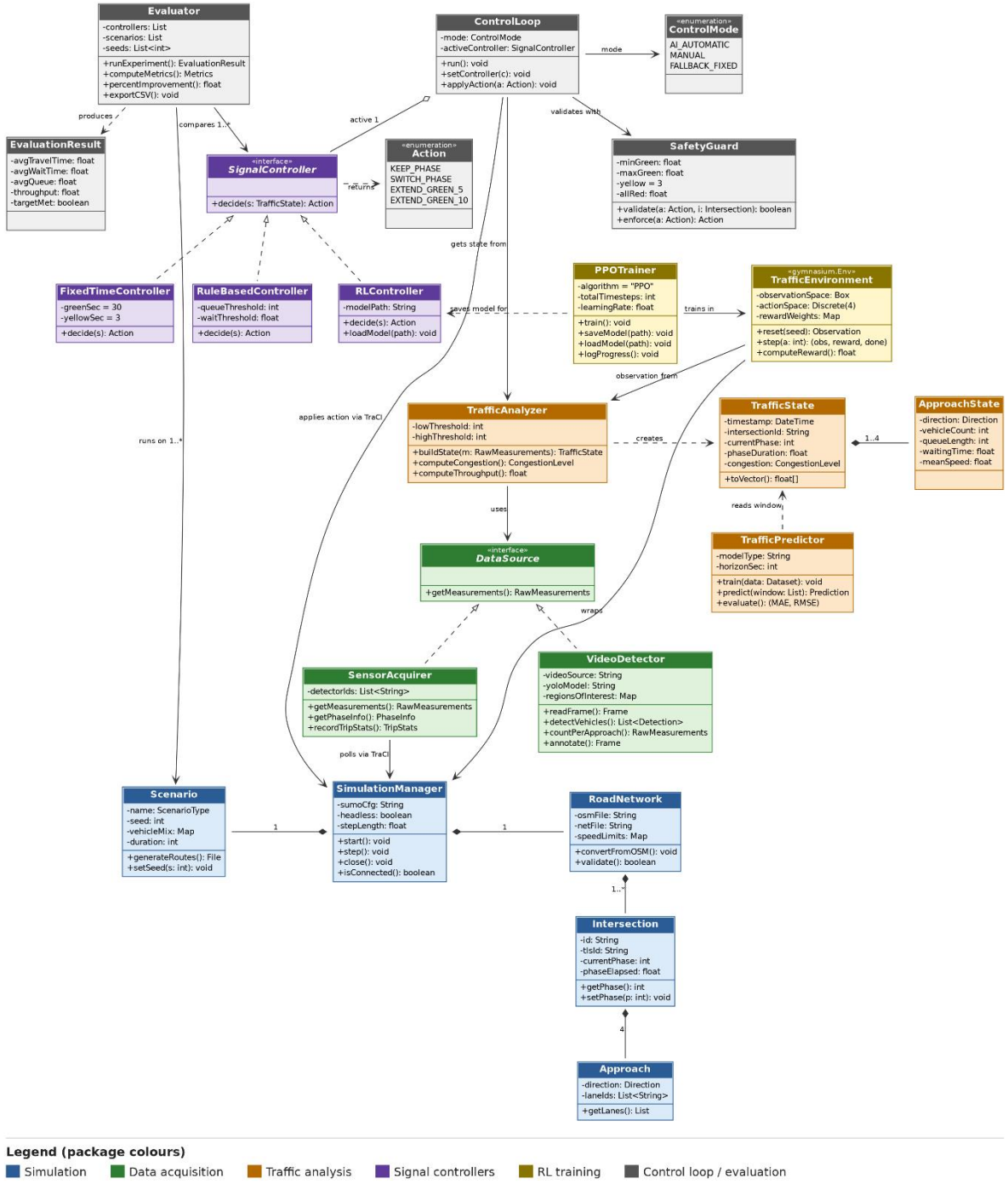


Figure B.1: Class Diagram of STMS

B.2 Use Case Diagram

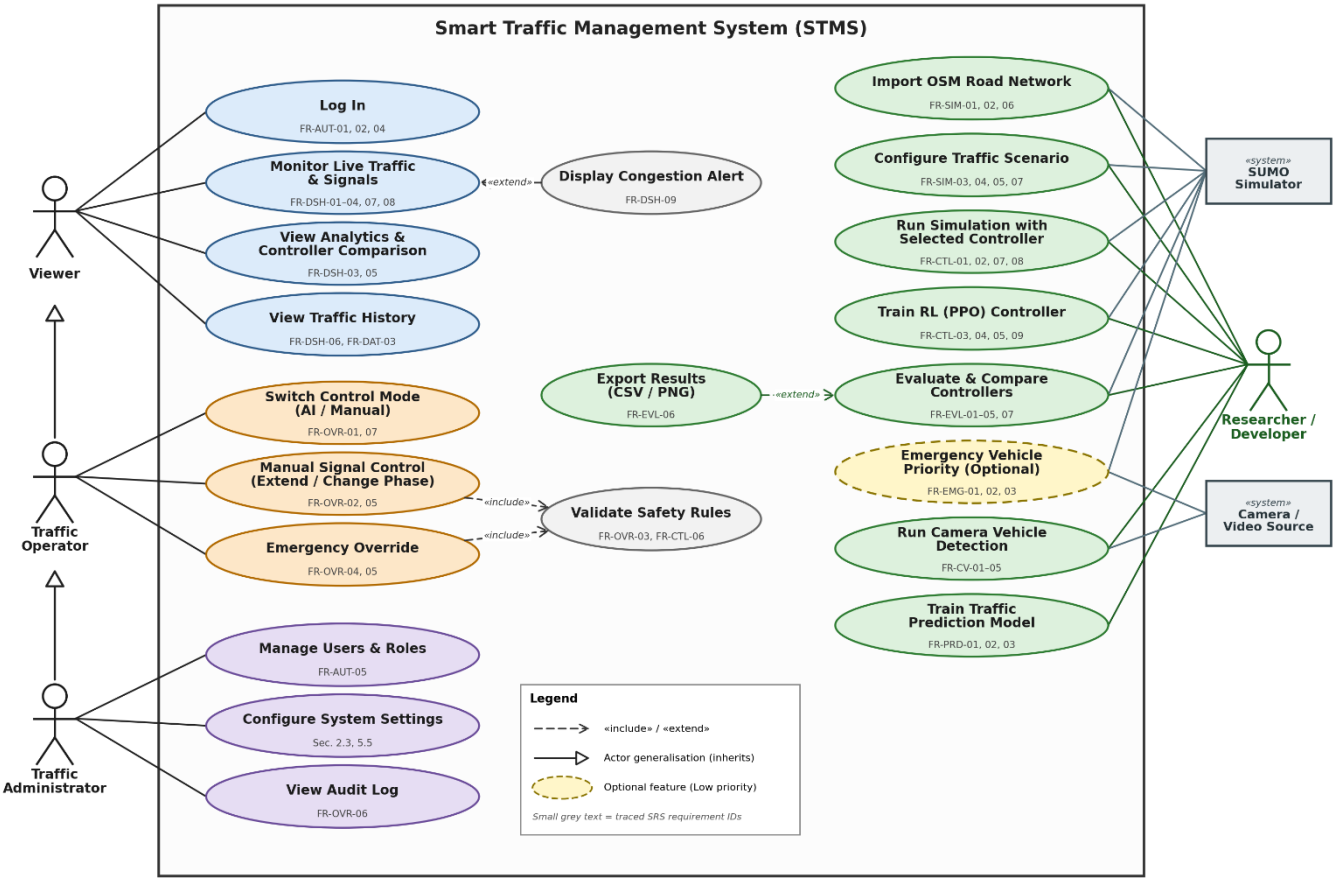


Figure B.2: Use Case Diagram of STMS

B.3 – Sequence Diagram(s) – AI control loop, manual override, login

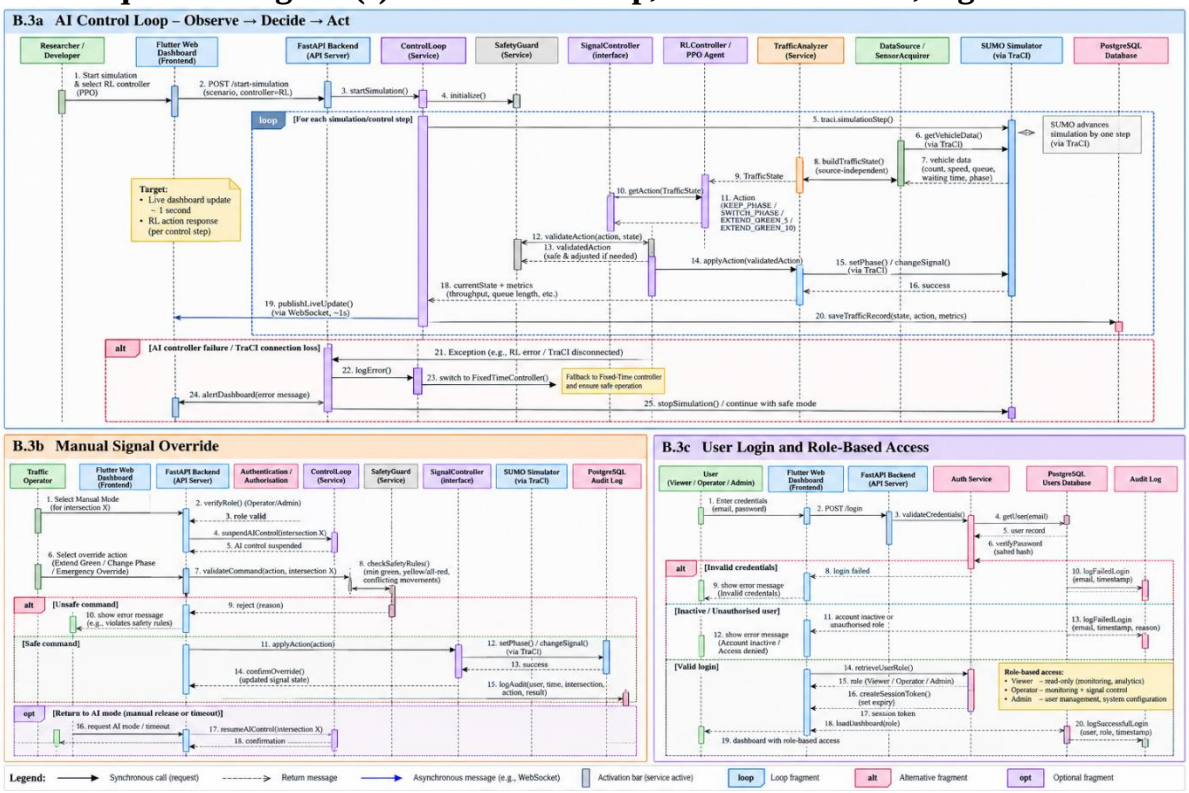


Figure B.3: Sequence Diagram of STMS

B.4 – Activity Diagram – training and evaluation workflow

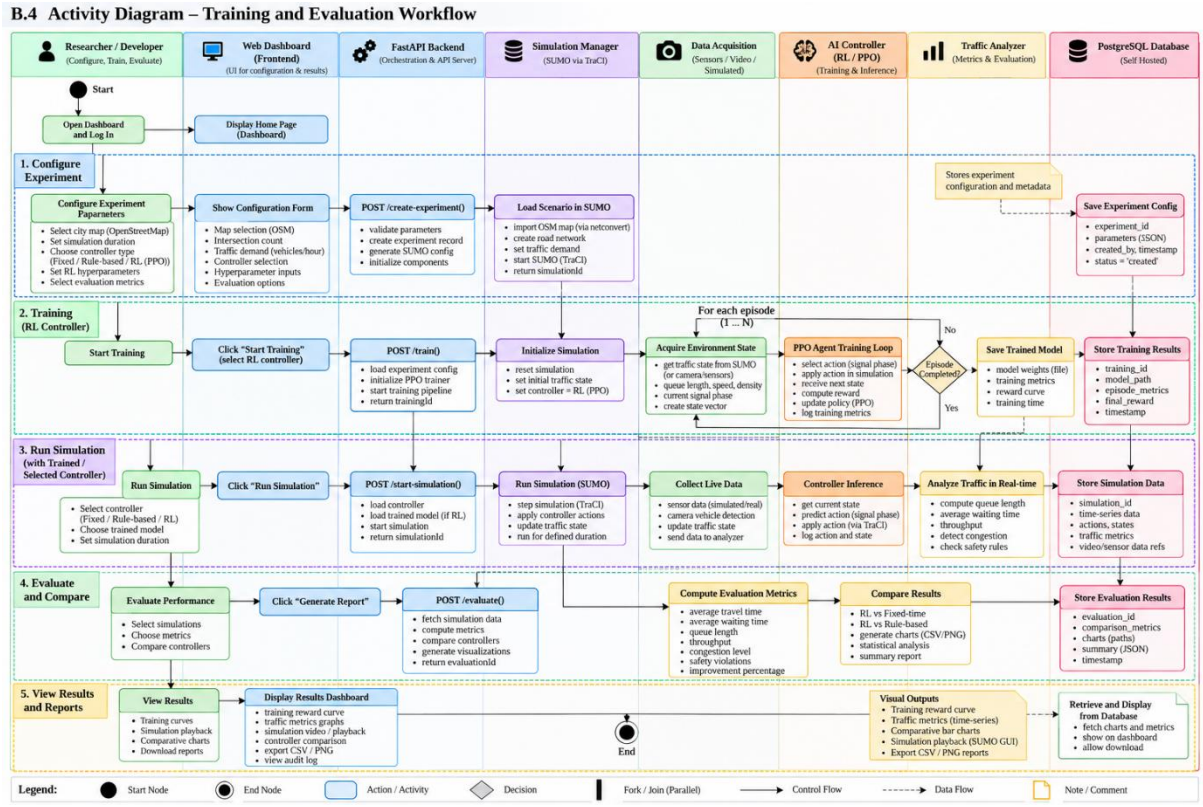


Figure B.4: Activity Diagram of STMS

B.5 – State Machine Diagram – traffic signal / control mode

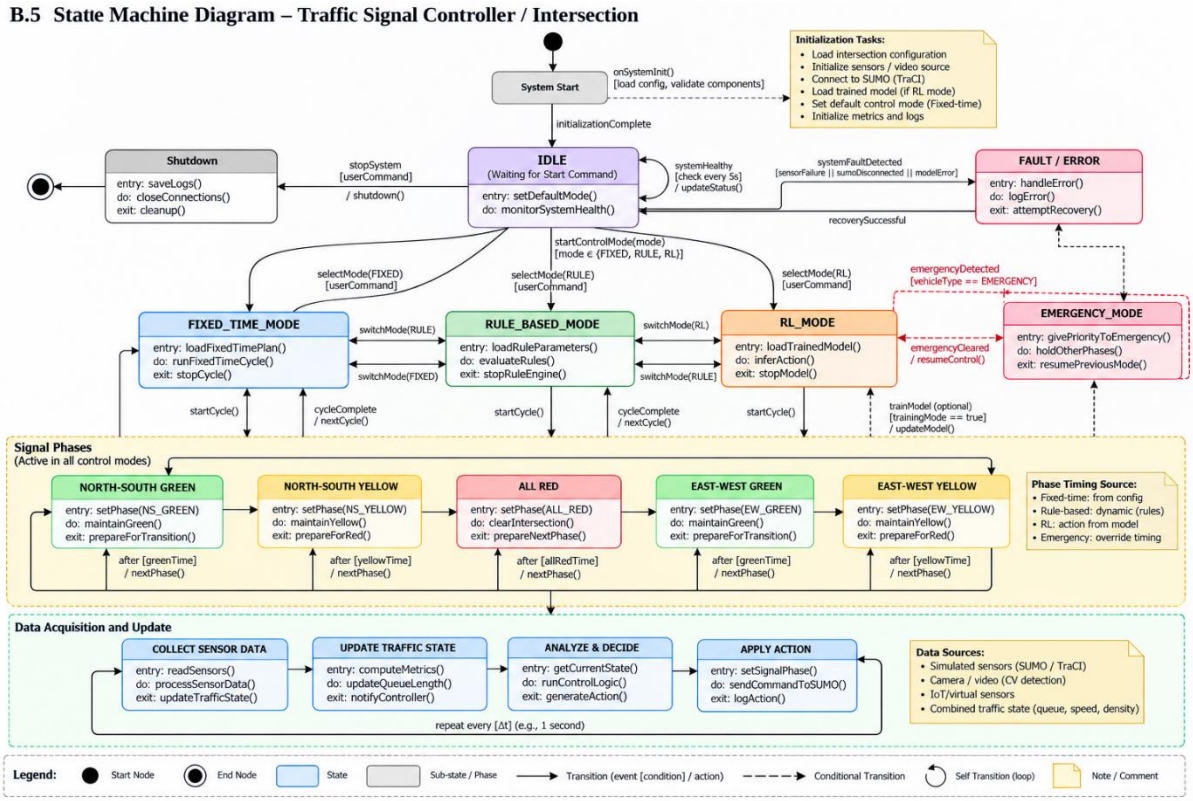


Figure B.5: State Machine Diagram of STMS

B.6 – Component Diagram

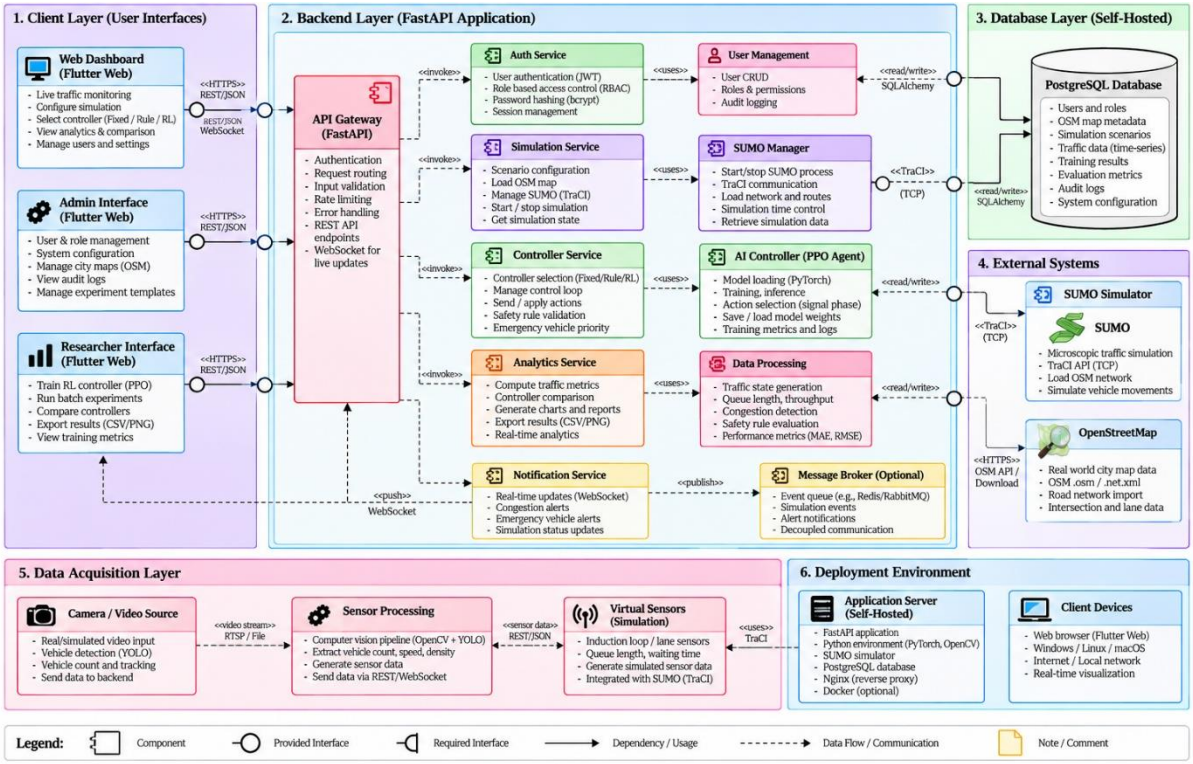


Figure B.6: Component Diagram of STMS

B.7 – Deployment Diagram

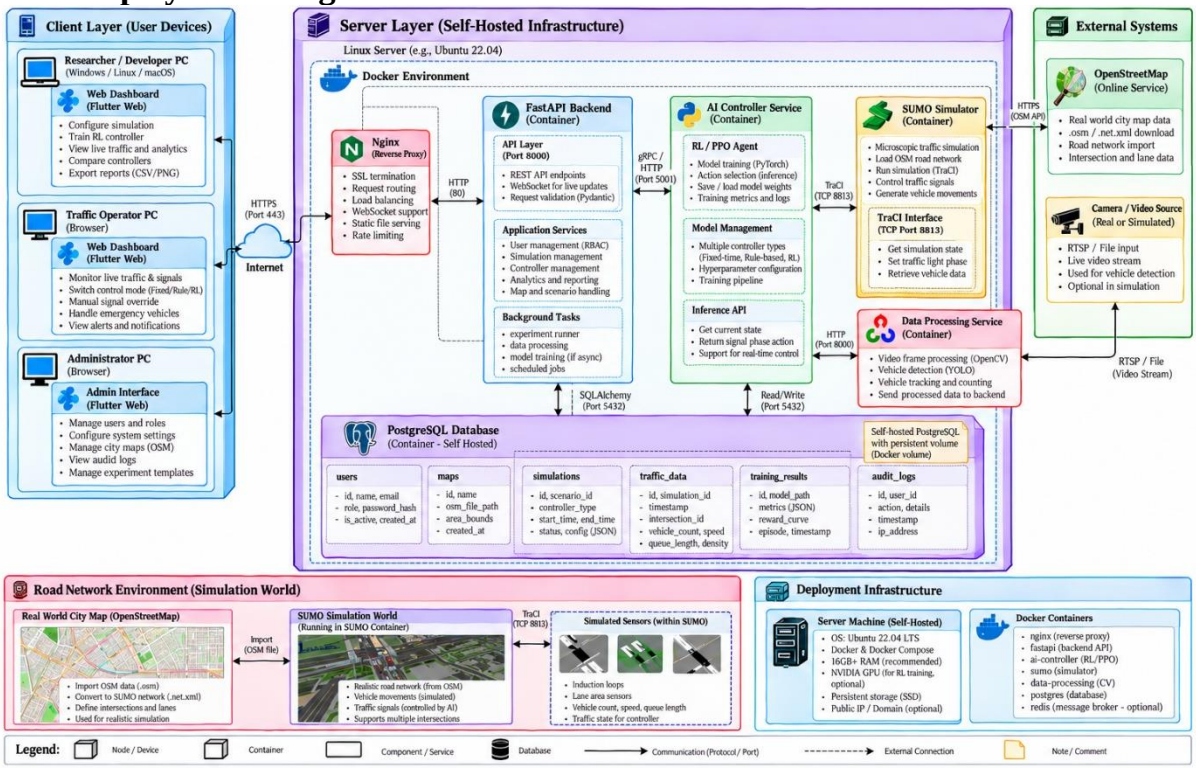


Figure B.7: Deployment Diagram of STMS

B.8 – Data Flow Diagrams (Level 0 and 1)

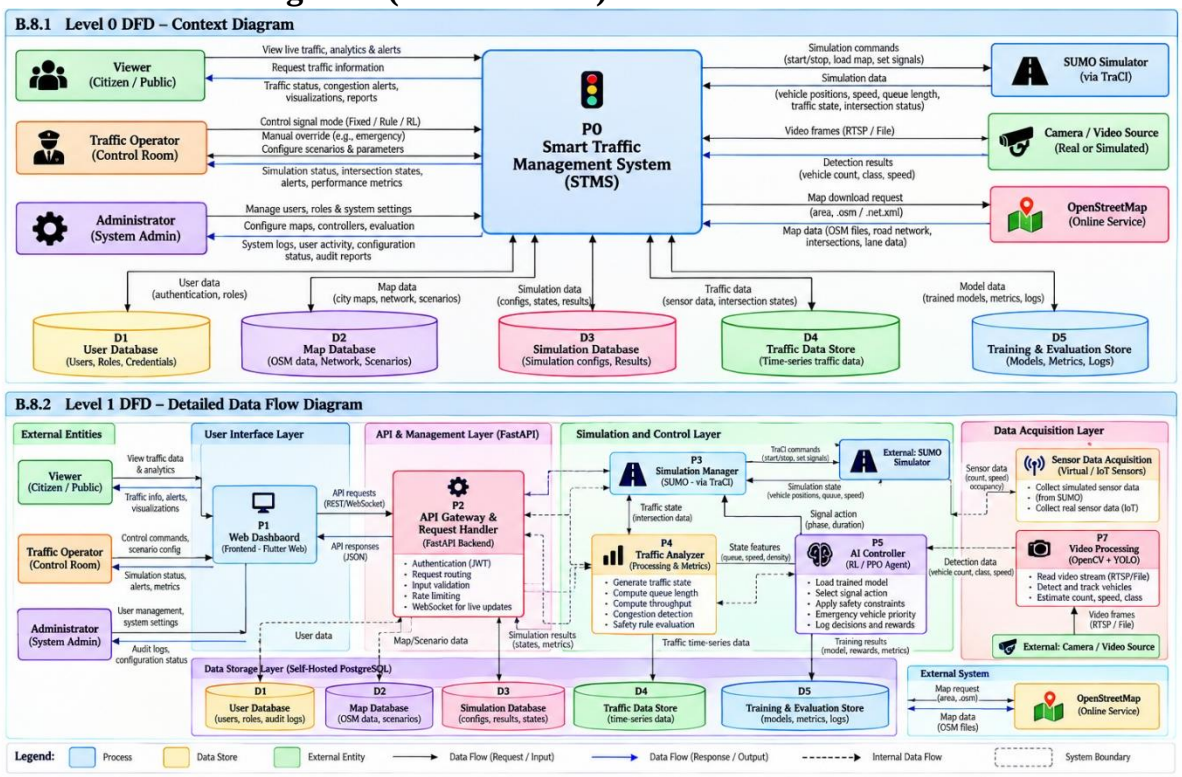


Figure B.8: Data Flow Diagram of STMS

B.9 – ER Diagram

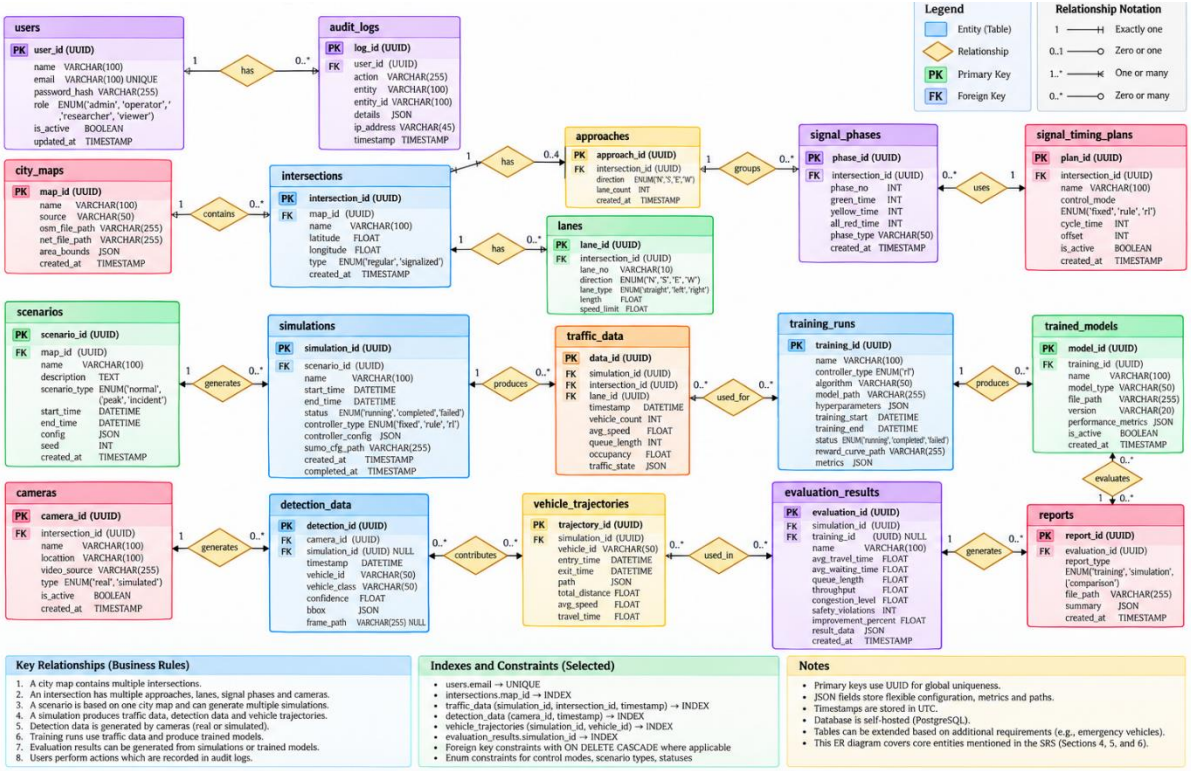


Figure B.9: ER Diagram of STMS